

Clinic Appointment & Patient Management System

Comprehensive Test Plan

Project: Programming 1 Summative Project

Group: 5

Team Members:

1. Anita Otoo
2. Samuel Anagbah
3. Stepy Rukundo
4. Chukwudumebi Ukogu

1. Test Objectives

The purpose of this document is to verify that all functional requirements of the Clinic Appointment System are met. Testing will cover Object-Oriented models, complex backend algorithms (time slot calculation), data validation loops, file persistence, and full system integration.

2. Unit Testing: Object-Oriented Models (models.py)

Test ID	Class / Method	Scenario Description	Test Input	Expected Output / Behaviour
TC-M01	User.verify_pin()	User attempts to log in with the correct PIN.	entered_pin = "1234", Object PIN = "1234"	Returns True.
TC-M02	User.verify_pin()	User attempts to log in with an incorrect PIN.	entered_pin = "0000", Object PIN = "1234"	Returns False.
TC-M03	Doctor.update_shifts()	Doctor updates working hours.	start = "08:00", end = "16:00"	Attributes shift_start_time and shift_end_time are updated accordingly.
TC-M04	Patient.update_profile()	Patient updates phone number	phone_number = "555-9999",	Provided attributes are

		and address.	address = "456 New St"	updated; unmodified attributes remain the same. Returns True.
TC-M05	Appointment.reschedule_appointment()	Appointment is shifted to a new date and time.	new_date = "2026-10-01", new_start_time = "14:00"	Date and start_time attributes are successfully overridden.
TC-M06	All Classes.to_dict()	Serialization prepares the object for JSON saving.	Call to _dict() on a Doctor object.	Returns a dictionary including base attributes and subclass attributes (e.g., "role": "Doctor", "specialization")

3. Business Logic Testing (operations.py)

Test ID	Method	Scenario Description	Test Input	Expected Output / Behaviour
TC-B01	generate_new_id_patient()	System generates an ID for a new registration.	List of existing patients.	Generates a random 8-digit integer and returns a string in the format "P-XXXXXXXXX".
TC-B02	find_record_by_id()	Search for an existing object by its ID.	search_id = "DR-201"	Returns the corresponding Doctor dictionary or object.

TC-B03	find_record_by_id()	Search for an ID that does not exist.	search_id = "A-999"	Returns None without crashing.
TC-B04	get_available_time_slots()	Calculate free slots for a doctor with no appointments.	shift = 09:00 - 12:00, appointments = []	Returns ["09:00 - 10:00", "10:00 - 11:00", "11:00 - 12:00"].
TC-B05	get_available_time_slots()	Calculate free slots when the doctor has a booked appointment.	shift = 09:00 - 12:00, booked at 10:00	Returns ["09:00 - 10:00", "11:00 - 12:00"]. The 10:00 slot is mathematically excluded.

4. Input Validation & Error Handling (validation.py)

Test ID	Scenario Description	Test Input	Expected Output / Behaviour
TC-V01	Empty string trap on Integer inputs (Menus).	Press Enter (Empty String)	Trapped. Prints "Oops sorry menu can not be left empty" and loops.
TC-V02	Invalid data type trap on Integer inputs.	Input = "two"	Trapped by ValueError. Prints "Invalid format" and loops.
TC-V03	Date validation (Birthdate).	Input = "2030-01-01" (Future date)	Trapped. Prints "Birth date cannot be after today's date" and loops.
TC-V04	Date validation (Appointment).	Input = "2020-01-01" (Past date)	Trapped. Prints "Appointment date cannot be before today's date" and loops.

TC-V05	Phone Number validation.	Input = "+2305551234"	Trapped. Rejects non-digits and invalid lengths. Requires exactly 8 digits starting with 5 or 7.
TC-V06	Optional Validation (Admin Updates).	Press Enter (Empty String)	Accepted. Breaks the validation loop to keep the current value unchanged.

5. Data Persistence (data_manager.py)

Test ID	Scenario Description	Test Input	Expected Output / Behavior
TC-D01	Application boot with missing JSON files.	Delete admins.json	FileNotFoundError is caught. System initializes admins = [] and boots normally without crashing.
TC-D02	System shutdown with mixed Object/Dict data.	Mixed list of freshly registered Patients (Objects) and old Patients (Dicts).	_to_serializable() converts objects to dicts; json.dump() successfully writes all data to .json files.

6. System Integration & Interface (main.py & sub_main.py)

Test ID	Scenario Description	Test Input	Expected Output / Behavior
TC-I01	Patient Self-Registration Flow.	Select "2", provide valid details.	Generates ID/PIN, creates Patient object, appends to list, displays credentials.
TC-I02	Login Routing.	user_id = "A-100",	Main router detects

		pin = "9999"	"A-" prefix, authenticates PIN, and correctly opens admin_menu.
TC-I03	Patient Booking Flow (End-to-End).	Select Doctor -> Select valid Date -> Select Slot.	Appointment object is created, ID is assigned, and it is appended to the master appointments list.
TC-I04	Doctor updates appointment status.	apt_id = "A301", new_status = "Completed"	Appointment status updates. Only allowed if apt.doctor_id matches the logged-in doctor.
TC-I05	Safe Shutdown.	Select "3" (Exit) from the main menu.	save_data() triggers, saving all session changes to JSON files before the program terminates.